Data Structures and Algorithms

Lab 09 - Binary Trees

Instructor: Sidrat-ul-Muntha

READ IT FIRST

Prior to start solving the problems in this assignments, please give full concentration on following points.

1. WORKING – This is individual lab. If you are stuck in a problem contact your teacher, but, in mean time start doing next question (don't waste time).

2. DEADLINE – One Week after assigned this.

3. SUBMISSION – This assignment needs to be submitted in a soft copy.

4. WHERE TO SUBMIT – Please visit your LMS.

5. **HOW TO SUBMIT** – All tasks must be completed in the **same Java file**, with Task 1's implementation reused in otter tasks. Clearly label Task 1, Task 2, and so on using comments in the code.

Learning Objectives

By the end of this lab, students will be able to:

- Create a binary tree and insert nodes as left and right children.
- Implement and demonstrate inorder, preorder, and postorder tree traversals.
- Count the total number of nodes in a binary tree.
- Determine the height of a binary tree.
- Search for an element in a binary tree.
- Apply recursive or iterative techniques to solve basic tree-related problems

---

Task 1: Create a Binary Tree

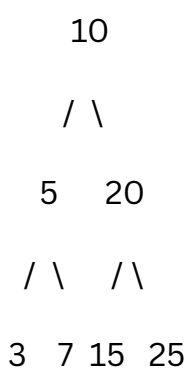
Write a Java program to implement a basic binary tree using a Node class. Each node should contain an integer value and references to its left and right children.

Implement the following:

- Node(data) – Create a new node with the given value.

- setLeft(parent, value) – Insert a new node as the left child of the specified parent if the position is empty.
- setRight(parent, value) – Insert a new node as the right child of the specified parent if the position is empty.

Demonstration
In the main() method, create the following binary tree:



Task 2: Inorder Traversal

Write a recursive method to traverse a binary tree using Inorder Traversal (Left → Root → Right).

Implement the following method:

- inorder(node) – Recursively visit the left subtree, display the current node, and visit the right subtree.

Demonstration

In the main() method, create the binary tree from Task 1 and display its inorder traversal.

Expected output:

Inorder: 3 5 7 10 15 20 25

Task 3: Preorder Traversal

Write a recursive method to traverse a binary tree using Preorder Traversal (Root → Left → Right).

Implement the following method:

- preorder(node) – Display the current node, then recursively visit the left and right subtrees.

Demonstration

In the main() method, create the binary tree and display its preorder traversal.

Expected output:

Preorder: 10 5 3 7 20 15 25

Task 4: Postorder Traversal

Write a recursive method to traverse a binary tree using Postorder Traversal (Left → Right → Root).

Implement the following method:

- postorder(node) – Recursively visit the left and right subtrees, then display the current node.

Demonstration

In the main() method, display the postorder traversal of the binary tree.

Expected output:

Postorder: 3 7 5 15 25 20 10

Task 5: Count Total Nodes in Binary Tree

Write a method that counts the total number of nodes in a binary tree.

Implement the following method:

- countNodes(node) – Return the total number of nodes in the tree.

Demonstration

In the main() method, create a binary tree containing several nodes and display the total number of nodes.

Expected output:

Total Nodes: 7

Task 6: Find the Height of a Binary Tree

Write a method to calculate the height of a binary tree.

For this task, define height as the number of levels, with an empty tree having height 0 and a leaf node having height 1.

Implement the following method:

- height(node) – calculate and return the height of the binary tree.

Demonstration

In the main() method, create a binary tree and display its height.

Expected output:

Height of Tree: 3

Task 7: Search for an Element in a Binary Tree

Write a method that searches for a given integer in a binary tree.

Implement the following method:

- search(node, key) – Return true if the element exists in the tree; otherwise, return false.

Demonstration

In the main() method, take an integer as input from the user and search for it in the binary tree.

Example:

Enter element to search: 15

Element Found

Enter element to search: 6

Element Not Found